COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	8/28/26	**DATE SUBMITTED:**	9/2//26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 6.1 Searching Techniques

1. PROCEDURE OUTPUT

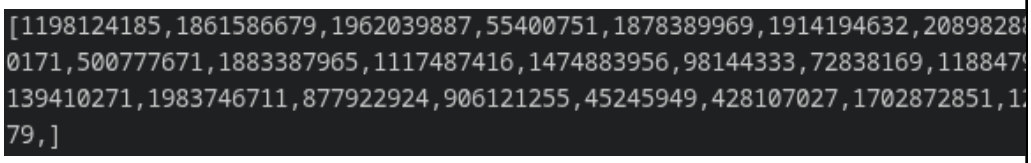
Screenshot	
Observation	The numbers generated are very large and look very random. These pseudorandom numbers are generated using the rand() function. Because generating these numbers still relies on an algorithm, they are not truly random. The algorithm uses the given “seed” to generate numbers that look random. The result of this is an array containing pseudo-random numbers.

Table 6-1. Data Generated and Observations

Code

```
 9   template<typename T>
10   void linearSearch(int size,T item,T array[]){
11       for (int i = 0; i < size;i++){
12           if(array[i] == item){
13               std::cout << "Data Found at index " << i << "\n";
14               return;
15           }
16       }
17       std::cout << "Data not found\n";
18   }
```

```cpp
const int max_size = 50;

void addRandomMembers(int dataset[]){
    srand(seed: time(timer: 0));
    std::cout << "[";
    for (int i = 0; i < max_size;i++){
        dataset[i] = rand();
        std::cout << dataset[i]<<",";
    }
    std::cout << "]\n";
}

int main(){

    int dataset[max_size];
    addRandomMembers(dataset);

    int itemToFind;
    std::cout << "Enter data to find:\n";
    std::cin >> itemToFind;

    linearSearch(size: max_size,item: itemToFind,array: dataset);
```

Output

```
~/Downloads/HOA/HOA#6
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" && g++ -std=c++20 main.cpp -o main -lncurses && "/hom
0171,500777671,1883387965,1117487416,1474883956,98144333,72838169,1188479414,707041105,159460
139410271,1983746711,877922924,906121255,45245949,428107027,1702872851,1232167990,525513924,7
79,]
Enter data to find:
1198124185
Data Found at index 0
```

```
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" && g++ -std=c++20 main.cpp -o main -lncurses && "/home/n
[1078954642,1008733196,112640083,58922460,461390634,1693563926,2086454023,2094108866,1438195654,
227,730380797,1574455160,1186386078,1029866854,1215518371,75649232,385312446,1351528579,19420885
75222132,1339628931,1004826539,821302410,1278599306,951451757,112014416,117154363,1058947647,339
41,]
Enter data to find:
24
Data not found
```

Observation

The code above uses a linear search algorithm to find a specified element inside an array. It takes in the size of the array, the array, and the item to be found. It uses a for loop to traverse the array. For each element, a comparison is made. If the comparison is evaluated to be true, it prints out a message saying that the element is found, then it returns early. If not, it will print out a message saying the element is not found. This algorithm has a best case of O(1) if the element required is found at the first element of the array, but has a worst case of O(n) when the element is in the last place or is not found.

Table 6-2a. Linear Search for Arrays

Code

```
46  template<typename T>
47  void linearSearchLL(Node<T> *headNode, T dataToBeFound){
48      while (headNode != nullptr){
49          if (headNode->data == dataToBeFound){
50              std::cout << "Found Data inside the node\n";
51              return;
52          }
53          headNode = headNode->next;
54      }
55      std::cout << "Data not found in nodes given\n";
56  }
```

```
    Node<char> *name1 = newNode(newData: 'N');
    Node<char> *name2 = newNode(newData: 'i');
    Node<char> *name3 = newNode(newData: 'e');
    Node<char> *name4 = newNode(newData: 'l');

    name1->next = name2;
    name2->next = name3;
    name3->next = name4;

    linearSearchLL(headNode: name1, dataToBeFound: 'l');
    deleteNodes(head: &name1);
    return 0;
```

Output

```
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" && g++ -std=c++20 main.cpp -
Found Data inside the node
```

Observation

The algorithm is the same as the one used on arrays. It loops through each element of the container. Because of that nature, it gives the algorithm a worst-case O(n) runtime if the element is found at the end or when not found at all.

Table 6-2b. Linear Search for Linked List

Code

```
20  template<typename T>
21  void binarySearchArr(T data[],int arrSize,T dataFind){
22      int low = 0;
23      int high = arrSize - 1;
24      int middle;
25
26      while (low <= high){
27          middle = (int)((low+high)/2);
28          if (dataFind == data[middle]){
29              std::cout << "The data was found\n";
30              return;
31          }
32          else if (dataFind < data[middle]){
33              high = middle - 1;
34          }
35          else {
36              low = middle + 1;
37          }
38      }
39
40      std::cout << "The data was not found\n";
41  }
```

```
int dataset[max_size] = {[0]=1,[1]=2,[2]=3,[3]=4,[4]=5,[5]=6,[6]=7,[7]=8,[8]=9,[9]=10};
std::cout << "Finding 10 inside the array\n";
binarySearchArr(data: dataset, arrSize: max_size, dataFind: 10);
```

Output

```
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" && g++ -std
Finding 10 inside the array
The data was found
```

Observation

The output utilizes binary search to find the given element. This algorithm tries to cut the search time in half by just looking at the left or right side of a given array. It goes for the middle of the array, then checks if it is greater than or less than the required number. If it is greater than the middle number, it checks the right half; if less than, it checks the left side. Because of this, the algorithm has a worst-case O(log n) runtime.

Table 6-3a. Binary Search for Arrays

Code

```cpp
template<typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* end){
    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != end && fast->next!= end){
        slow = slow->next;

        fast = fast->next->next;
    }

    return slow;
}

template <typename T>
void binarySearchLL(Node<T>* start, T dataFind) {
    Node<T>* last = nullptr;

    while (start != last) {
        Node<T>* middle = getMiddle(start, end: last);

        if (middle->data == dataFind) {
            std::cout << "Data found\n";
            return;
        }
        else if (middle->data > dataFind) {
            last = middle;
        }
        else {
            start = middle->next;
        }
    }

    std::cout << "Data not found\n";
}
```

```cpp

    Node<int> *num1  = newNode(newData: 10);
    Node<int> *num2  = newNode(newData: 20);
    Node<int> *num3  = newNode(newData: 30);
    Node<int> *num4  = newNode(newData: 40);
    Node<int> *num5  = newNode(newData: 50);
    Node<int> *num6  = newNode(newData: 60);
    Node<int> *num7  = newNode(newData: 70);
    Node<int> *num8  = newNode(newData: 80);
    Node<int> *num9  = newNode(newData: 90);
    Node<int> *num10 = newNode(newData: 100);

    num1->next  = num2;
    num2->next  = num3;
    num3->next  = num4;
    num4->next  = num5;
    num5->next  = num6;
    num6->next  = num7;
    num7->next  = num8;
    num8->next  = num9;
    num9->next  = num10;

    binarySearchLL(start: num1, dataFind: 60);
```

```cpp
    binarySearchLL(start: num1, dataFind: 55);
```

Output	~/Downloads/HOA/HOA#6 ❯ cd "/home/nielc/Downloads/HOA/HOA# Data found ~/Downloads/HOA/HOA#6 ❯ cd "/home/nielc/Downloa Data not found
Observation	To implement binary search on linked lists, where direct index access is impossible, a clever technique of using fast and slow moving pointers is used to find the middle. This approach, however, still uses an O(n) runtime, where n is the current number of sample space; Unlike in arrays, where accessing the middle index runs at O(1) runtime. Even though binary search can be implemented on a linked list, it is inefficient and defeats the purpose of using this search algorithm.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 10;

int main(){
    int myList[max_size] = {[0]=15,[1]=18,[2]=2,[3]=19,[4]=18,[5]=0,[6]=8,[7]=13,[8]=19,[9]=14};

    Node<int>* headNode = newNode(newData: myList[0]);
    Node<int>* tail = headNode;

    for (int i = 1; i < max_size; ++i) {
        tail->next = newNode(newData: myList[i]);
        tail = tail->next;
    }

    linearSearch(size: max_size, item: 18, array: myList);
    linearSearchLL(headNode, dataToBeFound: 18);
    deleteNodes(head: &headNode);

    return 0;
}
```

```cpp
template<typename T>
void linearSearch(int size,T item,T array[]){
    int comparisons = 0;
    for (int i = 0; i < size;i++){
        std::cout << "Comparisons made:" << ++comparisons << '\n';
        if(array[i] == item){
            std::cout << "Data Found at index " << i << "\n";
            return;
        }
    }
    std::cout << "Data not found\n";
}
```

```cpp
template<typename T>
void linearSearchLL(Node<T> *headNode, T dataToBeFound){
    int comparisons = 0;
    while (headNode != nullptr){
        std::cout << "Comparisons made:" << ++comparisons << '\n';
        if (headNode->data == dataToBeFound){
            std::cout << "Found Data inside the node\n";
            return;
        }
        headNode = headNode->next;
    }
    std::cout << "Data not found in nodes given\n";
}
```

```
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" && g++ -std=c++20 main.cpp -o main -lncurses && "/home/nielc/Downloads/HOA/HOA#6/"main
Comparisons made:1
Comparisons made:2
Data Found at index 1
Comparisons made:1
Comparisons made:2
Found Data inside the node
```

Figure 1: Problem 1 solution

Using sequential or linear search, the program is tasked with finding the first element with the same corresponding value as the given. To achieve this, the code for the linear search algorithm is used. This code is just modified to print how many comparisons it will take to find the first element the same as the given. Visually inspecting as a human, it can be said that it will take a total of 2 searches. This is proven by looking at the output of the code.

```
//================ ARRAY ======================================

template<typename T>
void linearSearch(int size,T item,T array[]){
    //int comparisons = 0;
    int count = 0;
    for (int i = 0; i < size;i++){
        //std::cout << "Comparisons made:" << ++comparisons << '\n';
        if(array[i] == item){
            std::cout << "Data Found at index " << i << "\n";
            std::cout << "The same element found: " << ++count << '\n';
            //return;
        }
    }
    if (count > 0) {
        return;
    }
    std::cout << "Data not found\n";
}
```

```
//============================== LINKED LIST ================================
template<typename T>
void linearSearchLL(Node<T> *headNode, T dataToBeFound){
    //int comparisons = 0;
    int count = 0;
    while (headNode != nullptr){
        //std::cout << "Comparisons made:" << ++comparisons << '\n';
        if (headNode->data == dataToBeFound){
            std::cout << "Found Data inside the node\n";
            std::cout << "The same element found: " << ++count << '\n';
            //return;
        }
        headNode = headNode->next;
    }
    if (count > 0) {
        return;
    }
    std::cout << "Data not found in nodes given\n";
}
```

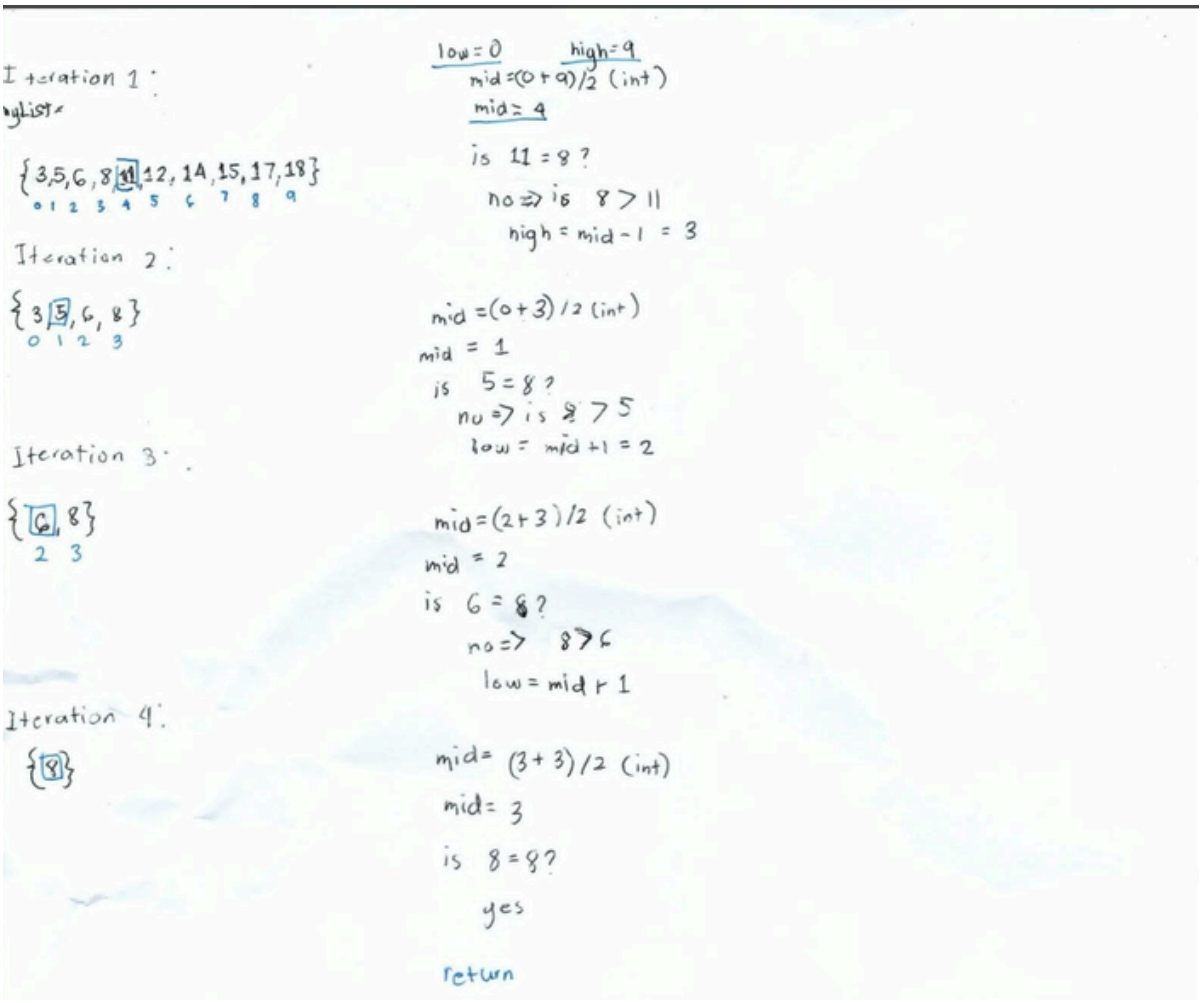
```
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" &&
Data Found at index 1
The same element found: 1
Data Found at index 4
The same element found: 2
Found Data inside the node
The same element found: 1
Found Data inside the node
The same element found: 2
```

Figure 2: Problem 2 Solution

The problem asks to modify the code so that it searches for all instances of a k-element, where k is the given element. To implement this, the exit condition when the element is found is removed and replaced by a line incrementing the variable that tracks the count of this k-element. This algorithm runs in O(n) runtime as it searches the array every time. Visually inspecting, we expect to find the answer to be 2. This answer is proven by running the code and looking at its output

```
int main(){
    int myList[max_size] = {[0]=3, [1]=5, [2]=6, [3]=8, [4]=11, [5]=12, [6]=14, [7]=15, [8]=17, [9]=18};

    binarySearchArr(arr: myList, arrSize: max_size ,dataFind: 8);
    return 0;
}
```




```
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" && g++ -std=c++20 main.c
Middle index: 4 , element: 11
less than middle, searching the left half, high: 3
Middle index: 1 , element: 5
greater than middle, searching the right half, low: 2
Middle index: 2 , element: 6
greater than middle, searching the right half, low: 3
Middle index: 3 , element: 8
The data was found
```

Figure 3: Problem 3 Solution

The diagram shows how the binary search cuts out each side of the array and checks the middle of the search space. Each iteration evaluating if its in the middle and searches the right or left half of the array. This way, the array can be searched without looking at each element. Cross-check the diagram from the output shows that it is correct.

```cpp
template<typename T>
void binarySearchArrRec(T arr[],int low,int high,T dataFind){
    if (low > high){
        std::cout << "Data not found"<<'\n';
        return;
    }
    int middle = (int)((low+high)/2);
    if (dataFind == arr[middle]){
        std::cout << "Element found at index: " << middle << '\n';
        return;
    }
    if (dataFind < arr[middle]){
        binarySearchArrRec(arr, low, high: middle - 1, dataFind);
    }
    else{
        binarySearchArrRec(arr, low: middle+1, high, dataFind);
    }
}
```

```
❯ cd "/home/nielc/Downloads/HOA/HOA#6/" && g++ -std=c++20
Element found at index: 3
```

Figure 4: Problem 4 solution

This problem asks to create code for the binary search algorithm; however, this time, the function needs to be recursive. Recursive is defined as “relating to or involving a program or routine of which a part requires the application of the whole, so that its explicit interpretation requires in general many successive executions.”, which, in layman's terms, means the function calls itself. Using the same algorithm and conditions as the non-recursive approach, it has the same “not found” condition; it declares the element as not found when the low is greater than the high. If it is not, it checks the middle of the search space. If it is found there, the function prints that it found the element, then exits. If not, it checks if it is lower or higher, then checks the left side or the right of the search space recursively by calling itself again, with different low or high values.

Conclusion:

In this activity, I coded the two most common searching algorithms: linear and binary search. Linear search traverses the whole array and compares each element. It is best used on unsorted or unsortable arrays or normal linked lists. This searching algorithm itself has an O(n) runtime complexity. For sorted arrays, binary search is highly recommended. This search algorithm cuts the search space in half every iteration; this gives this algorithm an O(log(n)) runtime, which can be significantly faster as input size increases. With the knowledge about these algorithms, I was able to solve the problems in the supplementary activity. I aim to use this newfound knowledge about search algorithms in my future projects as a programmer.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.